Produit

Ce handbook est la vue produit : la vision, le public, l'ensemble des fonctionnalités et la roadmap. Pour un tour concret de ce que fait l'app **aujourd'hui**, voir Comprendre Candex. Les éléments prospectifs sont tagués par statut, pour que le tableau reste honnête.

Le pitch

Candex transforme un tas éparpillé de canettes en une collection vivante et organisée qu'on garde dans sa poche. Découvre de nouvelles boissons, suis les canettes que tu possèdes, et vois où chacune se classe dans un dex mondial partagé par tous les collectionneurs.

À qui ça s'adresse

- **Les collectionneurs** — ceux qui gardent et font grandir une vraie collection de canettes et veulent la cataloguer correctement.
- **Les amateurs occasionnels** — ceux qui veulent se souvenir de ce qu'ils ont aimé et en trouver plus.
- **Les curieux** — tous ceux qui aiment parcourir boissons, marques et les détails derrière.

Ensemble des fonctionnalités

Légende des statuts : **Livré** (en prod) • **En cours** (en construction) • **Prévu** (sur la roadmap) • **Exploratoire** (direction produit, pas commencé).

Fonctionnalité	Ce qu'elle fait	Statut
Dex personnel	Débloquer des boissons et bâtir sa collection	Livré
Recherche & découverte	Parcourir le catalogue, ouvrir une boisson pour ses détails	Livré
World Dex	Une liste classée globale, avec historique de position	Livré
Détails d'une boisson	Noms, marque, saveurs, composition et claims par pays	Livré
Thèmes & langue	Light / Dark / System, anglais et français	Livré
Sécurité & backup	Connexion Firebase, et une collection sauvegardée	Livré
Scanner une canette	Ajouter une canette depuis une photo par reconnaissance d'image	En cours
Statistiques	Stats de collection — comptes, top marques, habitudes	Prévu
Communauté	Partager trouvailles, astuces et événements entre collectionneurs	Exploratoire

Le flux *Scanner une canette* est en partie en place côté API, qui fait déjà de l'analyse d'image et de couleur ; l'expérience de scan de bout en bout est encore en construction *(à confirmer côté app)*. *Statistiques* et *Communauté* reflètent la vision produit et ne sont pas livrées.

Roadmap

Éléments à court terme du board produit :

- **Édition de boisson** — éditer une boisson existante.
- **Suppression de boisson** — retirer une boisson.

- **Page canettes** — une vue dédiée aux canettes d'une boisson.
- **Déplacement de boissons** — déplacer la position d'une boisson (`dexPosition`).
- **CI/CD API** — finaliser le pipeline de livraison de l'API.

Principes produit

- **Solo, de bout en bout** — un seul ingénieur possède l'app, le dashboard et l'API ; les handbooks existent pour que ça ne devienne pas un goulot d'étranglement.
- **Bilingue par défaut** — chaque écran est livré en anglais et en français.
- **À toi, à ta façon** — thèmes et langue sont au choix de l'utilisateur, et sa collection est sauvegardée pour ne jamais être perdue.

Pour aller plus loin

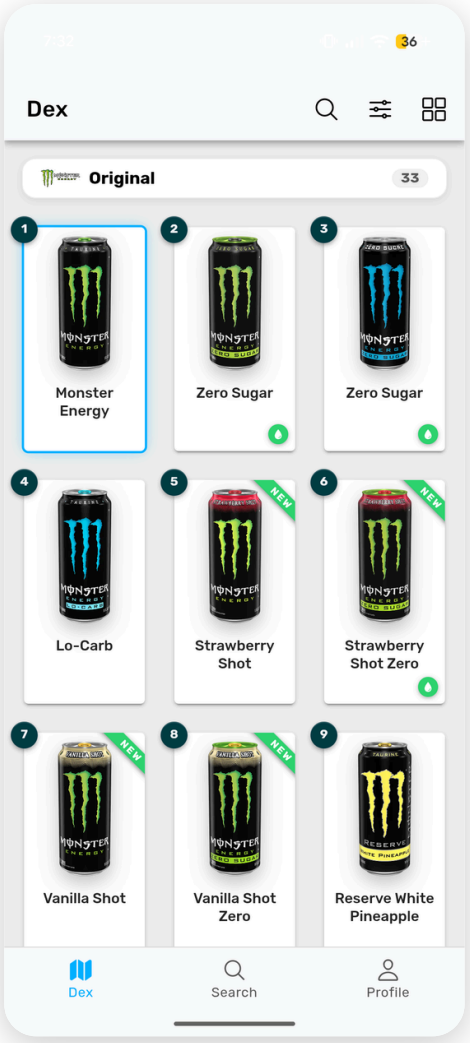
- Utiliser l'app aujourd'hui → Comprendre Candex
- Son apparence et son ressenti → UI / UX
- Comment c'est construit → Architecture

Comprendre Candex

Vous êtes en voyage, et sur une étagère vous repérez une boisson énergisante que vous n'avez jamais vue. L'avez-vous déjà goûtée ? Quelles éditions existent ? Candex transforme ce petit moment en jeu.

Ce qu'est Candex

Candex est une application mobile qui transforme la **découverte et la collection de boissons en jeu**. Vous trouvez des boissons du monde entier, marquez celles que vous avez goûtées, et complétez un catalogue personnel — votre **World Dex** — comme on complète une collection dans un jeu vidéo.



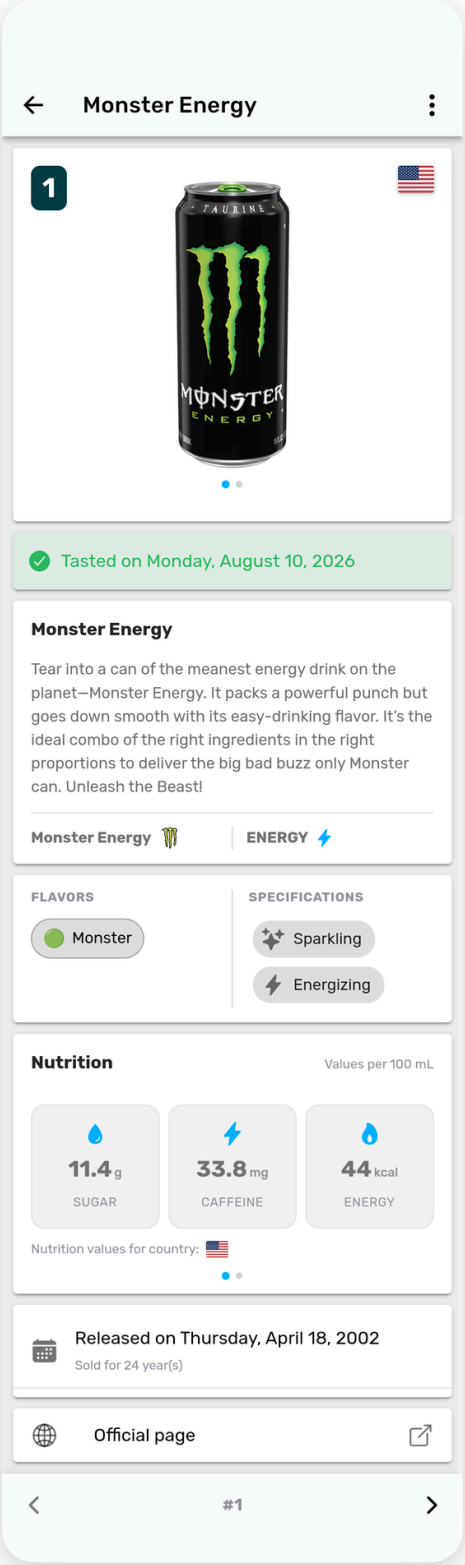
Votre World Dex — chaque boisson, remplie à mesure que vous la goûtez.

Toute l'application tient dans une boucle simple, en trois gestes :

1. **Découvrir** une boisson — parcourir le catalogue par marque, série et saveur.
2. **La goûter** — marquer une boisson comme goûtée ; elle s’illumine dans votre Dex, avec la date.
3. **Progresser** — voir votre World Dex se remplir, et ce qui vous manque encore.

À quoi ressemble une boisson

Chaque boisson a sa page : la canette, où et quand vous l’avez goûtée, une description, sa marque et sa catégorie, ses saveurs, et la nutrition pour 100 mL.



La page d'une boisson — goûtée, avec saveur, specs et nutrition.

Concepts clés

Quelques mots reviennent dans Candex. Les voici en clair ; le handbook **Glossaire / Domaine** (à venir) définira chacun précisément.

Terme	Signification
Drink	Une boisson du catalogue — ce que l'on collectionne.
Can	Une canette d'une boisson ; une boisson peut en avoir plusieurs (éditions, pays).
Brand	L'entreprise qui fabrique une boisson — Monster, Coca-Cola.
Series	Une gamme au sein d'une marque — par exemple la gamme *Original* de Monster.
Flavour	La saveur d'une boisson, affichée selon la langue.
World Dex	Votre catalogue de collection — ce que vous complétez.
Taste	Marquer une boisson comme goûtée, pour qu'elle compte dans votre Dex (aussi appelé *unlock*).

> **Note**
>
> D'anciennes notes produit mentionnent des idées comme *intake* ou *wishlist*. Elles ne font pas partie de l'application aujourd'hui ; elles sont donc laissées de côté ici et seront documentées si elles sont développées.

Pour qui

- **Explorateurs** — qui veulent découvrir des boissons locales, rares ou populaires ailleurs.
- **Collectionneurs** — qui veulent compléter leur World Dex et traquer ce qui leur manque.
- **Les curieux** — qui veulent juste un peu d'info en plus sur une boisson qu'ils ont en main.

Où aller ensuite

- **Ici pour travailler sur le code ?** → Travailler sur Candex
- Les définitions précises de chaque terme → Glossaire / Domaine *(à venir)*
- La vision produit et la roadmap → Produit *(à venir)*

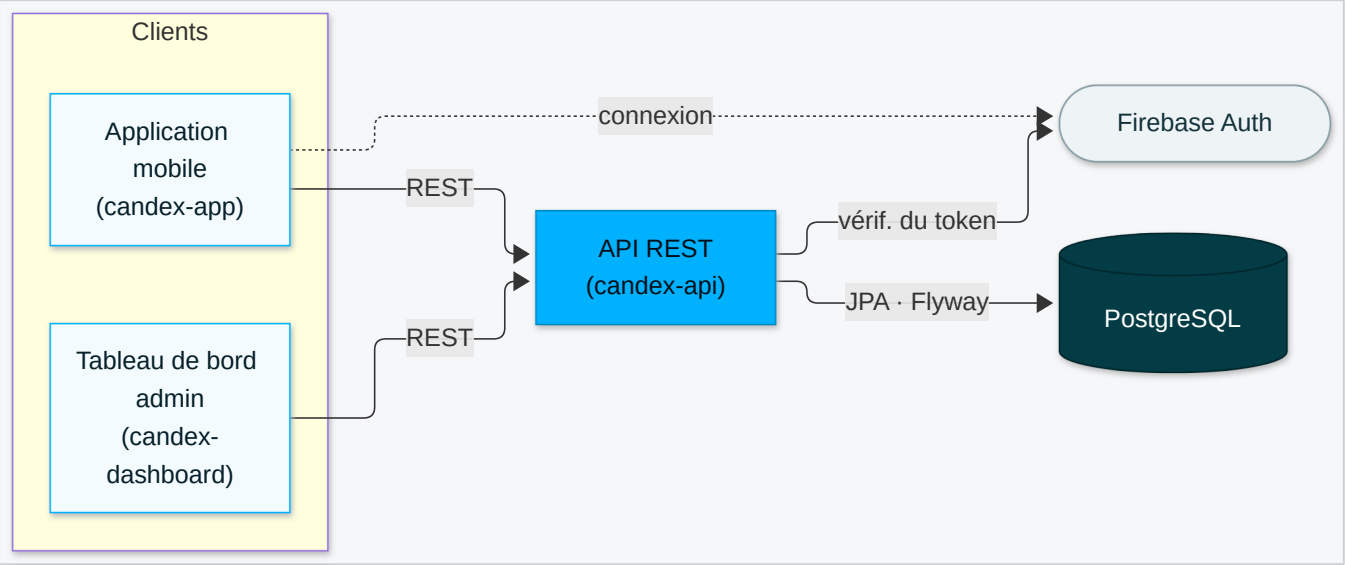
Votre première étape : le plus rapide pour comprendre Candex, c'est de l'utiliser — ouvrez l'app, trouvez une boisson que vous avez déjà bue et marquez-la comme goûtée. C'est votre première entrée dans le World Dex.

Travailler sur Candex

Cette page vous oriente si vous allez travailler sur Candex. Pour le produit lui-même — ce que c'est et l'expérience — commencez par Comprendre Candex.

Le système en un coup d'œil

Candex n'est pas une seule application mais plusieurs pièces qui fonctionnent ensemble. Les utilisateurs utilisent l'**application mobile**. Un **tableau de bord d'administration** garde le catalogue à jour. Les deux dialoguent avec une seule **API REST**, qui détient le domaine et stocke tout dans **PostgreSQL**. L'authentification est gérée par **Firebase**.



Dépôts

Dépôt	Rôle	Statut
`candex-app`	Client mobile (Angular + Ionic + Capacitor)	Actif
`candex-dashboard`	Tableau de bord d'administration du catalogue	Actif
`candex-api`	API REST et modèle de domaine, adossée à PostgreSQL	Actif
`candex-handbook`	Ce site de documentation (Astro Starlight)	Actif
`candex-crawler`	Crawler qui collecte les données de boissons et du catalogue	Actif
`candex-ai`	Module IA (Python)	Précoce / ébauche
`candex-backups`	Outils de sauvegarde Firestore	Déprécié — antérieur au passage à PostgreSQL
`candex-api-legacy`	Ancienne API (Node.js / Express)	Déprécié

> **Note**
>
> `candex-crawler` vit dans son propre dépôt (hors de l'espace de travail principal) ; son rôle est résumé ici d'après sa finalité et sera détaillé dans son propre handbook.

Stack technique

Application mobile — `candex-app`



- **TypeScript**
- **NgRx** — Store • Effects • Entity
- **RxJS**
- **Firebase** — Auth

Tableau de bord admin — `candex-dashboard`

- **Angular** — v20
- **Ionic** — v8
- **TypeScript**
- **Angular Material**
- **Firebase** — Auth

Back-end — `candex-api`

- **Spring Boot** — v3.5
- **Java** — 21
- **Maven** — build (wrapper)
- **PostgreSQL**
- **Flyway** — migrations
- **MapStruct** — mapping
- **Spring Security** — OAuth2
- **OpenAPI** — springdoc

Qualité & livraison

- **Docker** — Compose
- **JUnit** — + Mockito
- **Testcontainers**
- **JaCoCo** — couverture
- **ESLint**
- **Prettier**
- **Cypress** — e2e

Où vivent les choses

- **Domaine & API** — `candex-api`. Entités et règles métier dans `src/main/java` ; schéma dans PostgreSQL, qui évolue via les migrations Flyway sous `src/main/resources/db/migration`.
- **UI mobile** — `candex-app` (Angular + Ionic, NgRx pour l'état).
- **UI admin** — `candex-dashboard` (Angular + Ionic + Angular Material).

Le code est la source de vérité : quand la documentation et le code divergent, le code fait foi et la doc est corrigée.

L'installer en local

Candex tourne sous forme de trois applications — l'application mobile et le tableau de bord (Ionic/Angular, lancés avec npm) et l'API (Maven + Spring Boot, avec PostgreSQL), chacune dans trois environnements (local, dev, prod). L'installation complète — prérequis, clonage, fichiers d'environnement, base de données, et les commandes par application — vit dans le handbook **Local Installation**.

Où aller ensuite

- Configurer votre machine → Local Installation
- Comprendre l'assemblage et le pourquoi → Architecture & Vue d'ensemble
- Le langage précis du domaine → Glossaire / Domaine *(à venir)*

Votre première étape : clonez les dépôts, puis suivez Local Installation pour lancer l'API et au moins un front contre une base locale.

Installation locale

Ce guide fait tourner Candex sur votre machine : l'**API** avec une base PostgreSQL locale, l'**application mobile** et le **tableau de bord d'administration**. Si ce n'est pas déjà fait, parcourez d'abord Travailler sur Candex pour la vue d'ensemble.

La base locale est construite **de zéro** par les migrations Flyway de l'API (qui contiennent des seeds) : vous n'avez donc **aucun** besoin d'accès à un serveur.

Ce que vous aurez en route

Service	URL
API REST	`http://localhost:8080` (Swagger sur `/swagger-ui.html`)
Application mobile	`http://localhost:8100`
Tableau de bord admin	`http://localhost:8101`
PostgreSQL	`localhost:5432`

Prérequis

- **Git**
- **Node.js 20+** et **npm 10+** — pour l'app et le dashboard
- **JDK 21** — pour l'API (un wrapper Maven, `mvnw`, est inclus, donc une installation Maven séparée est optionnelle)
- **Docker** et Docker Compose — pour la stack locale Postgres + API
- La config d'un **projet Firebase** — pour l'authentification de l'app (voir la section app)
- **Android Studio** — optionnel, uniquement pour les builds natifs Android

Récupérer le code

Clonez les dépôts dans un même dossier :

```
git clone https://github.com/RusterX16/candex-api
git clone https://github.com/RusterX16/candex-app
git clone https://github.com/RusterX16/candex-dashboard
```

1 — API et base de données (`candex-api`)

Créez le fichier d'environnement à partir du modèle et renseignez les valeurs :

```
cd candex-api
cp .env.example .env     # puis éditez .env
```

Le chemin recommandé est le script d'aide, qui monte une stack conteneurisée jetable (Postgres 17 + API) depuis `compose.local.yaml` :

```
./scripts/candex.sh up        # démarre Postgres + API en local
```

Au démarrage, l'API exécute ses **migrations Flyway**, qui créent le schéma et insèrent les données de base (marques, versions d'app, etc.) — une base fonctionnelle sans aucun accès serveur.

Autres commandes utiles :

```
./scripts/candex.sh down      # arrête la stack (données conservées)
./scripts/candex.sh reset     # efface le volume de la base locale
./scripts/candex.sh psql      # ouvre psql sur la base locale
./scripts/candex.sh logs      # suit les logs de l'API
```

L'API est alors sur `http://localhost:8080`, avec Swagger UI sur `/swagger-ui.html`.

> **Sans Docker**
>
> Si vous préférez votre propre PostgreSQL, définissez `CANDEX_DB_URL`, `CANDEX_DB_USERNAME` et `CANDEX_DB_PASSWORD`, puis démarrez l'API avec `./run-dev.sh` (ou `./mvnw spring-boot:run -Dspring-boot.run.profiles=dev`).

2 — Application mobile (`candex-app`)

Installez les dépendances :

```
cd candex-app
npm install
```

L'app génère son environnement Angular à partir de fichiers env (`scripts/generate-env.js`). Créez-les à partir du modèle :

- `.env.dev` et `.env.prod` — un projet Firebase chacun, plus `API_URL` (copiez `.env.example`)
- `.env.local` — juste `API_URL=http://localhost:8080`, qui écrase l'URL de l'API pour les runs locaux

Config Firebase. L'app s'authentifie via Firebase : `.env.dev` / `.env.prod` ont besoin des sept valeurs `FIREBASE_*`, et le build Android a besoin de `android/app/google-services.json`. Récupérez-les depuis votre projet Firebase (Paramètres du projet → votre app web) ou auprès du owner du projet. Aucune valeur Firebase n'est incluse dans cette documentation.

Lancez contre votre API locale :

```
npm run start:local     # http://localhost:8100 (Firebase dev + API localhost)
```

`start:local` s'appaire avec le profil **dev** de l'API — gardez les deux en `dev`. Pour un build natif Android (nécessite Android Studio) : `npm run android:dev`.

3 — Tableau de bord admin (`candex-dashboard`)

```
cd candex-dashboard
npm install
cp .env.example .env     # renseignez API_URL_LOCAL et les identifiants admin
npm start                # http://localhost:8101
```

Tout assembler

Pour un setup local cohérent, gardez ensemble le mode **local** de l'app et le profil **dev** de l'API. Ordre typique : démarrez l'API (`candex.sh up`), puis le dashboard (`npm start`) et l'app (`npm run start:local`).

Dépannage

- **Port déjà utilisé** — arrêtez le processus qui occupe `8080`, `8100`, `8101` ou `5432`, ou changez le port.
- **L'API n'atteint pas la base** — vérifiez que Docker tourne et que `candex.sh up` rapporte un `db` sain ; contrôlez les valeurs `DEV_DB_*` dans `.env`.
- **Le build de l'app échoue avec « Missing env vars »** — `generate-env.js` a besoin de `API_URL` et des sept valeurs `FIREBASE_*` dans le fichier `.env.*` chargé.
- **Base vide** — les migrations s'exécutent au démarrage de l'API ; vérifiez `candex.sh logs` pour confirmer que Flyway les a appliquées.

Où aller ensuite

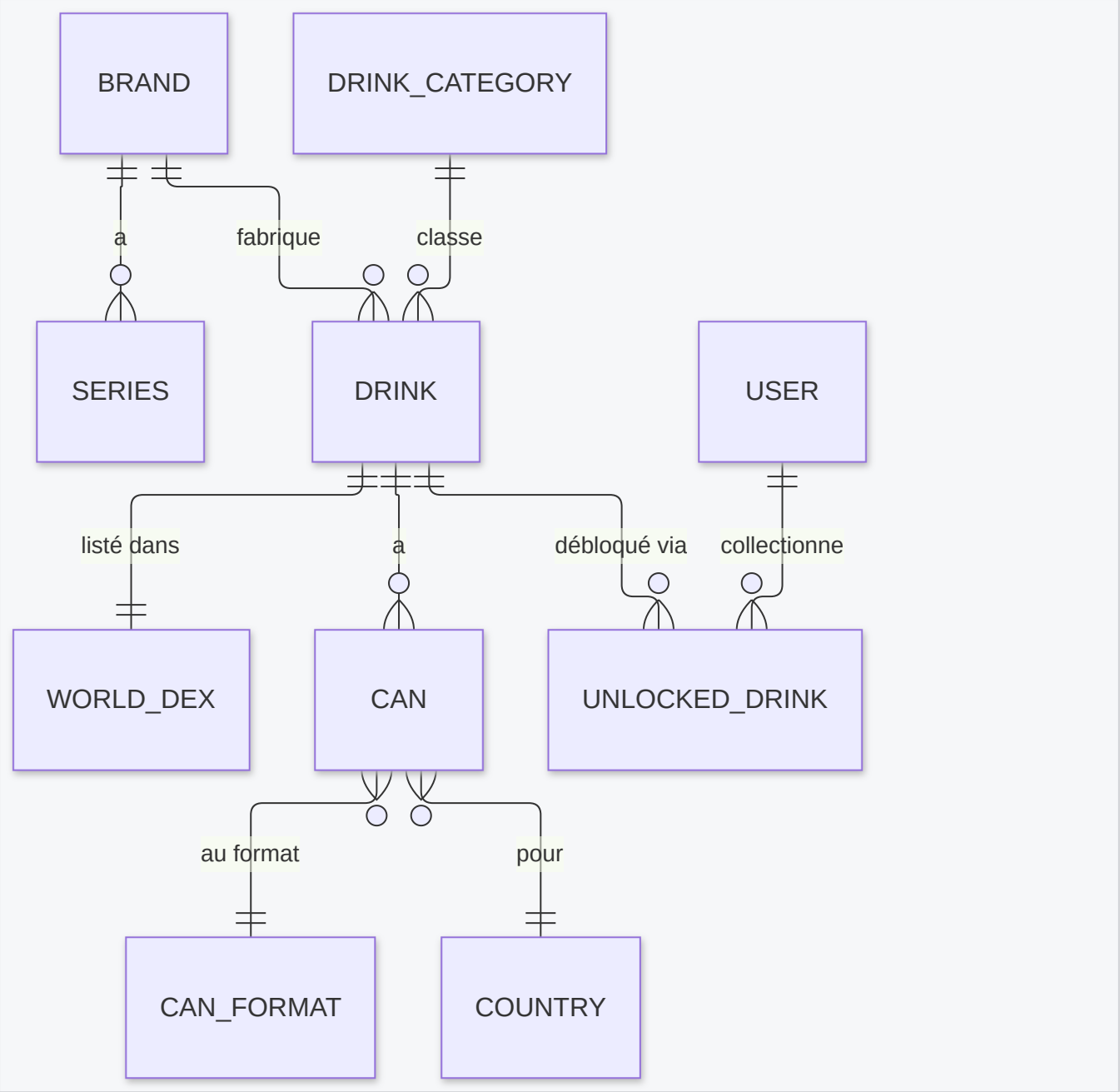
- Internes de l’API → Back-end *(à venir)*
- Environnements et releases → Déploiement *(à venir)*

Architecture & Vue d'ensemble

Voici la carte de la construction de Candex. Elle reste haut-niveau et renvoie ailleurs : les handbooks **Back-end**, **Front-end**, **Base de données** et **Déploiement** approfondissent chaque domaine. Pour les briques et les dépôts, voir Travailler sur Candex.

Modèle de domaine

Le catalogue est un graphe d'entités autour de **Drink**. Une boisson appartient à une **marque** et à une **catégorie**, est listée une fois dans le **World Dex**, et possède une ou plusieurs **canettes** (par pays, format et couleurs). Les utilisateurs construisent leur collection en **débloquant** des boissons.



Autour de ce cœur gravitent des entités de support — `Flavor` (avec catégories et traductions), `Color`, `Language`, `DrinkLocalization`, et `WorldDexArrival` / `WorldDexHistory` pour l'ordonnancement. Les entités de catalogue (`Brand`, `Series`, `Flavor`, `CanFormat`) portent un lien **`introduced_in_version`** vers `AppVersion` : le catalogue est donc versionné avec l'app. Le modèle complet est dans le handbook **Base de données & Modèle**.

Architecture de l'API

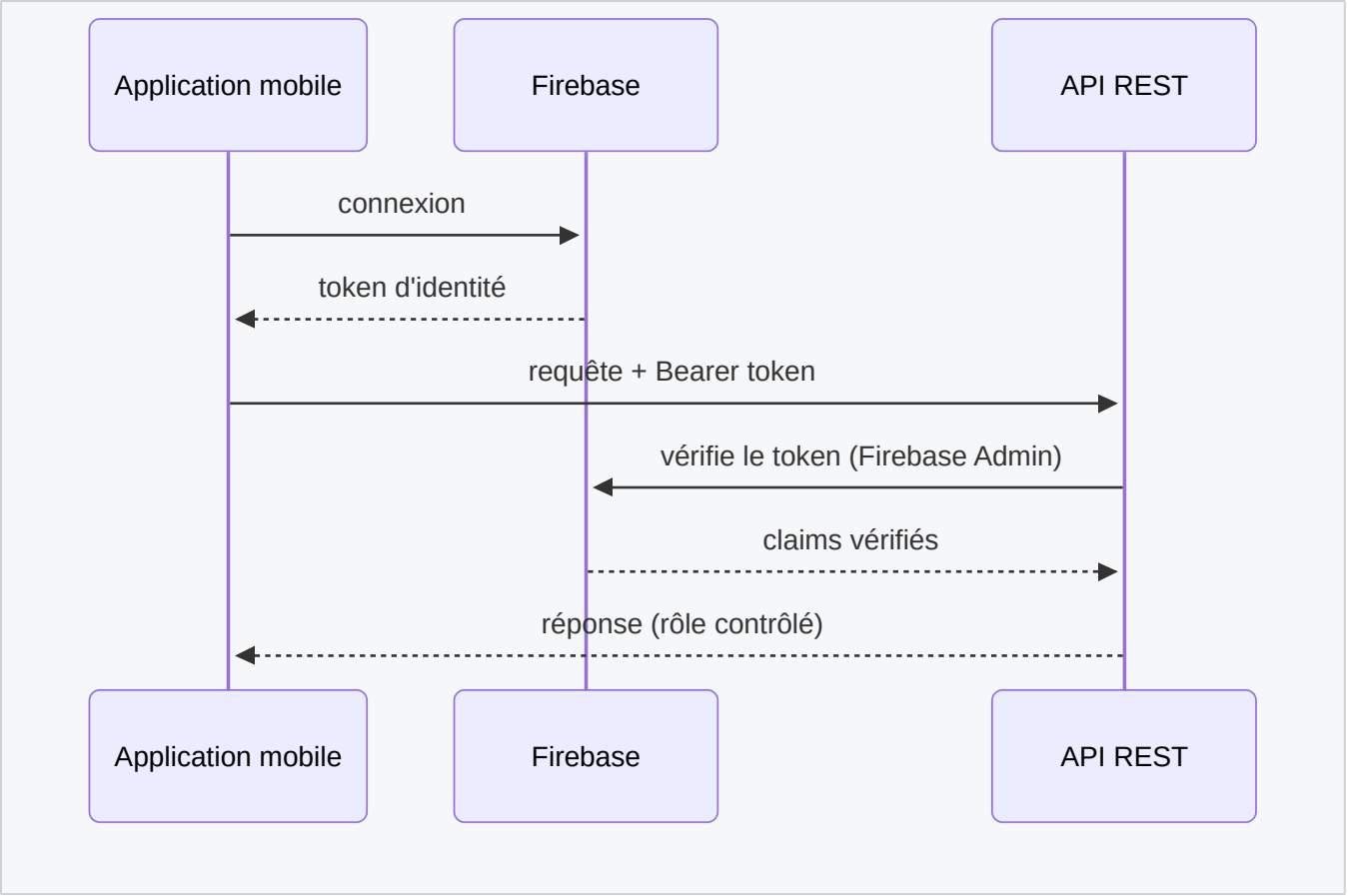
`candex-api` est une application Spring Boot en couches, sous `dev.ruster.candex` :

Couche	Package	Responsabilité
Web	`controller`	Endpoints REST
Contrat	`dto` (`request` / `response`), `mapper`	Formes de l'API, mappées vers/depuis les entités avec MapStruct
Métier	`service` (`core`, `analysis`)	Logique de domaine
Données	`repository`, `entity`	Spring Data JPA sur PostgreSQL
Transverse	`security`, `exception`, `validation`, `config`, `job`	Auth, erreurs, validation, config, tâches planifiées

La frontière `dto` + `mapper` découple le contrat de l'API du modèle JPA — les contrôleurs n'exposent jamais les entités directement.

Authentification & sécurité

L'app se connecte via **Firebase** ; l'API vérifie le token Firebase à chaque requête (`FirebaseAuthenticationFilter` / `FirebaseAuthenticationService`, via le SDK Firebase Admin) et autorise par rôle (`ROLE_ADMIN`, `ROLE_USER`).



Données & migrations

Le domaine vit dans **PostgreSQL**. Le schéma est détenu par les migrations **Flyway** sous `candex-api/src/main/resources/db/migration` : chaque changement est un fichier versionné `V__*.sql`, et les données de seed (marques, versions d'app, etc.) sont aussi livrées en migrations — c'est pourquoi une base neuve est directement utilisable. Un rôle SQL en lecture seule est provisionné par migration pour une inspection sûre.

Les front-ends

Les deux front-ends sont en Angular + Ionic et dialoguent avec la même API REST.

- **Application mobile** (`candex-app`) — état géré avec **NgRx**, organisé en slices : `auth`, `dex`, `drink`, `user`, `preferences`, `app`.
- **Tableau de bord admin** (`candex-dashboard`) — Angular + **Angular Material**, pour garder le catalogue à jour.

Voir le handbook **Front-end** pour la structure et les conventions.

Topologie de déploiement

Chaque environnement est une stack Docker Compose d'un conteneur **API** et d'un conteneur **PostgreSQL 17** :

Environnement	Fichier compose	Notes
local	`compose.local.yaml`	Stack jetable via `scripts/candex.sh` (voir Installation locale)
dev	`compose.dev.yaml`	`api-dev` + `db-dev` ; API sur l'hôte `8081`, debug `5005`
prod	`compose.prod.yaml`	`api-prod` + `db-prod`

> **À confirmer**
>
> Le TLS et le reverse-proxy ne sont pas dans ces fichiers compose, bien qu'une variable `CADDY_EMAIL` existe — le point d'entrée public (probablement **Caddy**) semble donc géré au niveau serveur. Le handbook **Déploiement** documente la topologie exacte.

Notes de conception

Groundé depuis le code et la config ; les points marqués *à confirmer* nécessitent ton apport.

- **API en couches avec frontière DTO/mapper** — les packages `dto` + `mapper` séparent le contrat public des entités JPA, pour que le modèle de données évolue sans casser les clients.
- **Firebase pour l'identité, PostgreSQL pour le domaine** — l'authentification est déléguée à Firebase tandis que toutes les données de catalogue et de collection sont relationnelles dans Postgres (visible depuis les dépendances et la config).
- **Schéma détenu par Flyway** — la base n'est jamais éditée à la main ; chaque changement est une migration versionnée, ce qui rend aussi l'install locale reproductible.

- **Tableau de bord séparé** — la gestion du catalogue est une app distincte de l'app mobile grand public. *(Rationale à confirmer.)*
- **Catalogue versionné** — les entités de catalogue référencent `AppVersion`. *(Usage visé à confirmer.)*

Pour aller plus loin

- Le modèle de données → Base de données
- Les couches de l'API → Back-end
- Le front-end → Front-end

Base de données & Modèle

Voici la référence des données de Candex. Le domaine vit dans **PostgreSQL**, et le schéma est entièrement détenu par les migrations **Flyway**. Pour la vue d'ensemble et le cœur simplifié, voir Architecture & Vue d'ensemble.

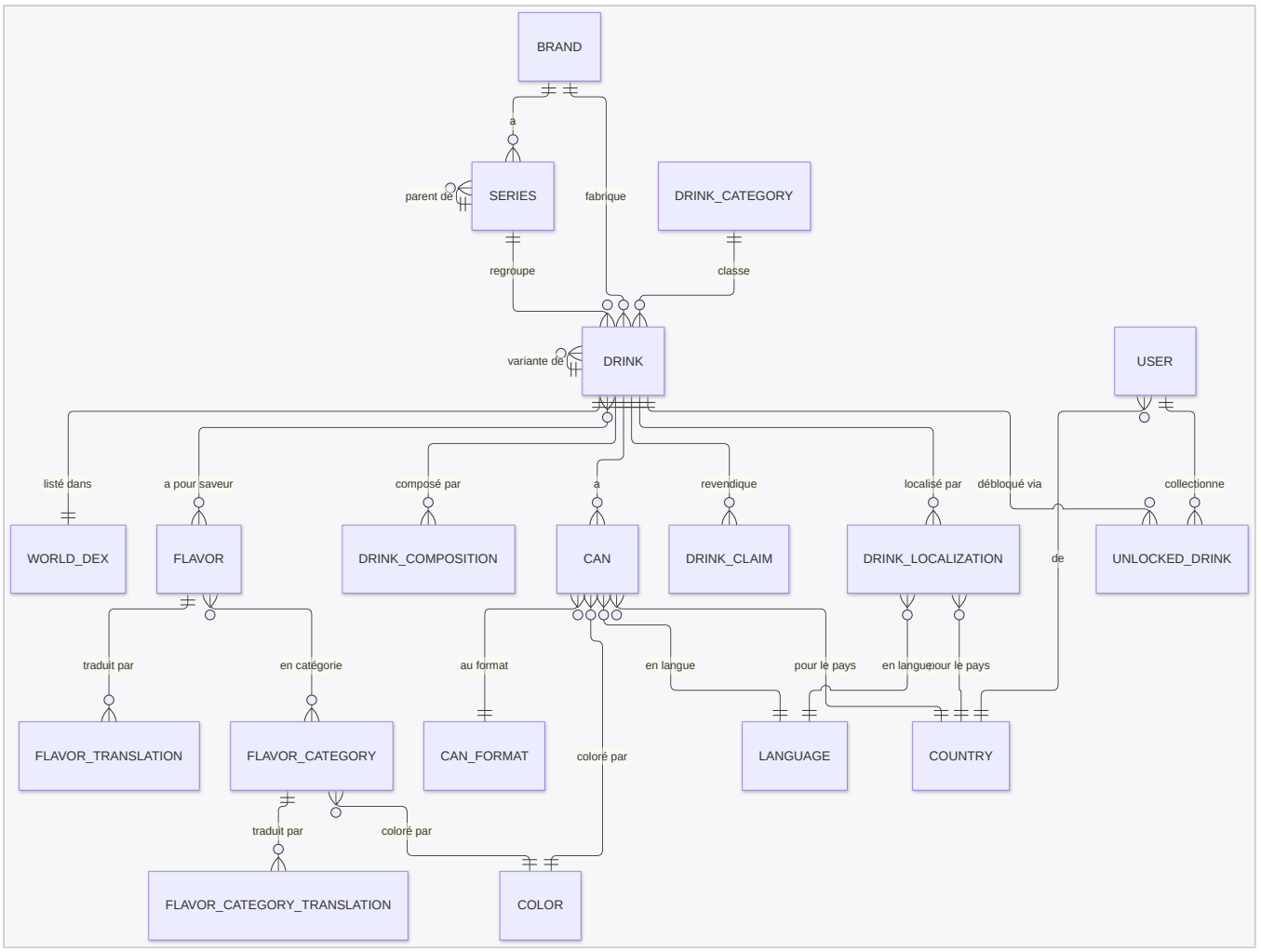
Organisation du schéma

Le schéma de base est posé par les migrations `V1`–`V6`, en trois domaines plus la mécanique runtime :

Migration	Domaine
`V1` — core reference schema	Tables de référence : pays, langues, couleurs, catégories, versions d'app
`V2` — user data schema	Utilisateurs et leurs boissons débloquées
`V3` — catalogue & dex schema	Marques, séries, boissons, canettes, saveurs, World Dex
`V4` — runtime routines	Fonctions PostgreSQL
`V5` — runtime triggers	Triggers qui maintiennent l'ordre du World Dex
`V6` — static reference data	Seeds des tables de référence

Le modèle d'entités

Le catalogue est un graphe autour de **Drink**. Une boisson appartient à une marque, une série et une catégorie ; possède des canettes, localisations, compositions et revendications ; a une ou plusieurs saveurs ; et est listée une fois dans le World Dex. Les utilisateurs collectionnent les boissons en les débloquant.



Quelques points que le diagramme abstrait :

- **Catalogue versionné.** La plupart des entités de catalogue et de référence (`Brand`, `Series`, `Flavor`, `Color`, `Country`, `Language`, `DrinkCategory`, `CanFormat`, `User`) portent un lien `introduced_in_version` vers `AppVersion` : chaque ligne est rattachée à la version d'app qui l'a introduite. Ces arêtes sont omises du diagramme pour rester lisible.
- **Tables de jointure à clé composite.** `UserUnlockedDrink` (user + drink), `DrinkLocalization` (drink + country + language), `FlavorTranslation`, `FlavorCategoryTranslation` et `DrinkCountry` utilisent des clés primaires composites (voir `entity/id`).
- **Ordonnancement World Dex.** `WorldDexArrival` et `WorldDexHistory` suivent l'ordre et l'arrivée des entrées ; cet ordre est maintenu dans la base (voir plus bas).

Données de référence & seeds

Les tables de référence (pays, langues, couleurs, catégories de boisson, formats de canette) et les premières versions d'app sont seedées par migrations, donc une base neuve est directement utilisable. Les baselines de marques et séries (par exemple Monster et Coca-Cola) sont seedées dans des migrations ultérieures (`V13`, `V14`). Comme le catalogue est versionné, les lignes de référence et de catalogue sont introduites contre une `AppVersion` précise.

Migrations & Flyway

Flyway détient le schéma (`spring.flyway.enabled=true`, `locations=classpath:db/migration`).

- `V1`–`V6` sont la **baseline canonique** pour les bases vides ; les bases non vides existantes sont baseline à la version `6`.
- Tout changement ultérieur est une migration **incrémentale** `V7+`.
- Les migrations appliquées sont **immuables** — ne jamais renommer, réordonner ou éditer une migration après déploiement.
- Les scripts snapshot legacy sont conservés sous `src/main/resources/sql/archive/`.

Pour changer le schéma ou les données de référence, ajoutez le prochain fichier `V<n>__courte_description.sql` ; il s'exécute automatiquement au prochain démarrage de l'API.

Ordonnancement du World Dex

Le World Dex est un catalogue **ordonné**, et cet ordre est maintenu **dans la base** plutôt que dans l'app. Des routines PostgreSQL (`V4`) et des triggers (`V5`) décalent les entrées lors des insertions ou suppressions de boissons, afin que les positions restent contiguës et correctes. Cette logique a été affinée au fil de plusieurs migrations (`V9`, `V12`, `V17`–`V20`) ; les fichiers de migration font foi pour le comportement exact. Le handbook **Back-end** couvre la façon dont l'API l'expose.

Console SQL en lecture seule

La migration `V16` provisionne un **rôle de base en lecture seule** pour une inspection sûre. Son mot de passe vient du placeholder Flyway `readonly_password` (`CANDEX_SQLCONSOLE_READONLY_PASSWORD`). Ce rôle alimente la console SQL du tableau de bord admin, pour exécuter des requêtes sans accès en écriture.

Conventions

- Tables et colonnes en `snake_case` ; les clés étrangères sont `<entité>_id` (ou un code naturel comme `country_code`, `language_code`).
- Les tables de jointure et de traduction utilisent des **clés primaires composites**, définies dans `entity/id`.
- Les migrations sont **versionnées et immuables** ; la base n'est jamais éditée à la main.
- Les lignes de catalogue et de référence sont rattachées à une `AppVersion` via `introduced_in_version`.

Pour aller plus loin

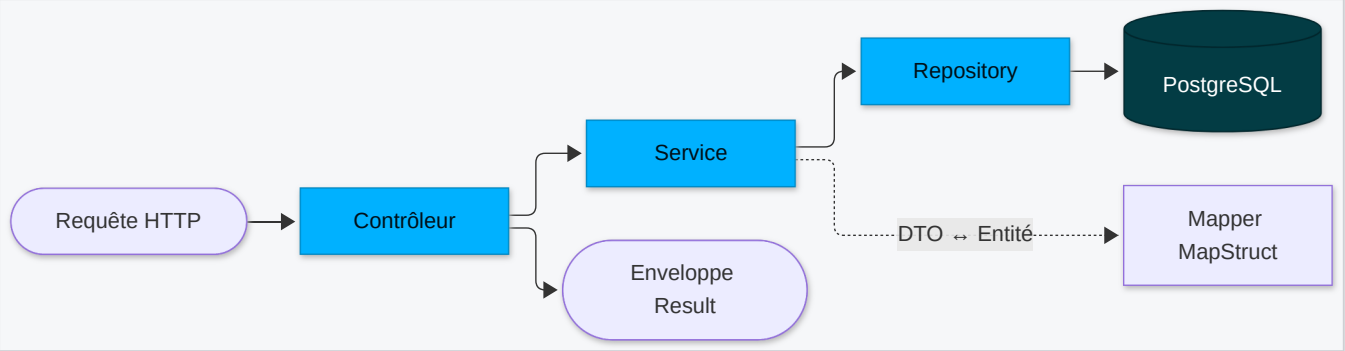
- Les couches qui l'utilisent → Back-end
- Les termes derrière les tables → Glossaire
- Comment elle est sauvegardée → Backups

Back-end

`candex-api` est une application Spring Boot en couches. Ce handbook couvre comment elle est construite ; pour le domaine qu'elle sert, voir Base de données & Modèle, et pour le système autour, Architecture & Vue d'ensemble.

Flux d'une requête

Chaque requête traverse les mêmes couches. Les contrôleurs renvoient une enveloppe typée **`Result`** ; les services portent la logique ; les repositories sont du Spring Data JPA ; et la frontière entre entités JPA et formes de l'API est franchie par des mappers **MapStruct**.



Chaque couche a exactement un rôle :

Couche	Responsabilité	Ne doit pas
Contrôleur	Associer une requête HTTP à une méthode, contrôler l'accès (`@PreAuthorize`), renvoyer un `Result`. Mince.	Porter de la logique métier ni toucher la base
Service (`service/core`)	La logique métier : orchestration, transactions, vérification des règles, appel des repositories, mapping entités ↔ DTO, levée d'exceptions.	Connaître quoi que ce soit du HTTP
Repository	Accès Spring Data JPA à un agrégat — requêtes dérivées et personnalisées.	Porter des règles métier
Entité	Le modèle de domaine JPA (état persisté).	Franchir la frontière du service
DTO (`dto/request`, `dto/response`)	Les formes d'entrée et de sortie de l'API.	Porter des préoccupations de persistance
Mapper (MapStruct)	Convertir entités ↔ DTO à la frontière du service.	—
Transverse (`security`, `exception`, `validation`, `config`, `job`, `result`)	Auth, erreurs structurées, validation, configuration, jobs planifiés, enveloppe de réponse.	—

La règle d'or : les entités n'atteignent jamais le contrôleur, et les contrôleurs ne portent jamais de logique.

L'enveloppe de réponse

Chaque endpoint renvoie un `Result<T>` générique :

```
{ "success": true, "data": { "...": "..." }, "error": null }
```

- `data` porte la charge utile en cas de succès ; `error` porte une erreur structurée en cas d'échec ; `success` est le drapeau.
- Les réponses de collection ajoutent un en-tête **`X-Total-Count`** ; les créations ajoutent un en-tête **`Location`**.

Surface de l'API

Les chemins des ressources sont centralisés dans `ApiPaths` et exposés sous la base de l'API (par exemple `/api/drinks`). Environ **45 endpoints** répartis sur ~14 ressources (28 `GET`, 9 `POST`, 4 `DELETE`, 3 `PUT`, 1 `PATCH`) :

Ressource	Chemin
Drinks	`/drinks` (dont import en masse `import`, `import/diff`, `import/sync`)
Cans • Can formats	`/cans` • `/can-formats`
Brands	`/brands`
Flavors • Flavor categories	`/flavors` • `/flavor-categories`
Colors • Drink categories	`/colors` • `/drink-categories`
World Dex	`/world-dex`
Users • Unlocked drinks	`/users` • `/user-unlocked-drinks`
App versions • Reports	`/app-versions` • `/reports`
Console SQL admin	`/admin/...`

La doc interactive est sur **Swagger UI** (`/swagger-ui.html`). Les contrats exacts requête/réponse relèvent du handbook **API / Contrats**.

Autorisation

Les endpoints sont gardés par `@PreAuthorize` et `SecurityExpressions` :

Règle	Pour
`USER_OR_ADMIN`	Lectures authentifiées (catalogue, World Dex)
`ADMIN_ONLY`	Écritures du catalogue et opérations admin
`PERMIT_ALL`	Les rares endpoints publics
`@securityAccess.canAccessUserId(...)`	Données propres à l'utilisateur — chacun ne touche que ses propres enregistrements

L'identité vient de Firebase (voir Architecture & Vue d'ensemble).

Services & logique métier

La logique métier est dans `service/core` — un service par ressource (`DrinkService`, `BrandService`, `WorldDexService`, `UserService`, …) — plus des services de support : `ImageService` (analyse), `UploaderService` (média), `ReportService` et `SqlConsoleService`.

Gestion d'erreurs

Chaque exception étend un `CustomException` commun, qui **porte son propre `HttpStatus`** et se sérialise dans le champ `error` de l'enveloppe `Result` (un

`dto/response/Error` contenant le message et le code de statut). Les exceptions se répartissent en deux familles.

Erreurs API (`exception/api`) — problèmes de requête et de données :

Exception	HTTP	Levée quand
`ElementNotFoundException`	404	Une entité référencée n'existe pas
`InvalidFieldValueException`	400	Un champ a un format ou une valeur invalide
`MissingRequiredFieldException`	400	Un champ requis est absent
`UnsupportedFieldException`	400	Un champ inconnu/non supporté est utilisé
`InvalidImportPayloadException`	400	Un payload d'import en masse est malformé
`SqlExecutionException`	400	Une requête de la console SQL échoue
`ImmutableFieldModificationException`	409	Un champ immuable est modifié après création
`UniqueConstraintViolationException`	409	Une contrainte d'unicité est violée
`MissingConfigurationException`	422	Une configuration requise (ex. une version d'app) manque

Erreurs métier (`exception/business`) — violations de règles de domaine :

Exception	HTTP	Levée quand
`MissingRequiredCanException`	400	Une boisson est enregistrée sans aucune canette
`MissingRequiredLocalizationException`	400	Une boisson est enregistrée sans aucune localisation
`InvalidDexPositionException`	400	Une position du World Dex est hors limites
`InvalidEditionConfigurationException`	400	Une édition / variante est mal configurée
`DrinkAlreadyHasPredecessorException`	409	Une boisson est liée comme prédécesseur à deux boissons
`ImageAnalysisException`	422	L'analyse d'image échoue

En bref : les exceptions `api` décrivent ce qui ne va pas dans la requête ; les exceptions `business` décrivent quelle règle métier a été enfreinte.

Validation

Les DTO de requête sont validés avec Bean Validation, câblé via `ValidationConfig`. Une entrée invalide ressort en erreur `api` plutôt que d'atteindre la couche service.

Jobs planifiés

Deux jobs de fond tournent périodiquement :

- `UnverifiedUserCleanupJob` — supprime les utilisateurs n'ayant jamais vérifié leur compte.
- `WorldDexHistoryPruneJob` — élague l'ancien historique du World Dex.

Configuration

La configuration vit dans `config` et dépend du profil (`dev` / `prod`) : `OpenApiConfig` (Swagger), `CorsConfig` (origines autorisées), `JacksonConfig` (JSON), `SqlConsoleConfig` et `ValidationConfig`. Pour la lancer en local, voir Installation locale ; les spécificités d'environnement relèvent du handbook **Déploiement**.

Conventions

- Les contrôleurs renvoient `ResponseEntity<Result<T>>` ; les chemins viennent des constantes `ApiPaths`.
- Les entités ne quittent jamais la frontière du service — les DTO (`dto/request`, `dto/response`) la franchissent via les mappers MapStruct.
- Chaque endpoint déclare son accès avec `@PreAuthorize`.
- Les erreurs sont des exceptions structurées (`api` / `business`) exposées via l'enveloppe `Result`.

Pour aller plus loin

- Les données qu'il persiste → Base de données
- Les endpoints qu'il expose → API / Contrats
- Comment il est sécurisé → Sécurité
- Comment il est testé → Tests

Front-end

Candex a deux front-ends Angular + Ionic qui consomment le même Back-end : l'**app mobile** que tiennent les utilisateurs, et le **tableau de bord admin** qui garde le catalogue à jour. Pour le système autour, voir Architecture & Vue d'ensemble.

App mobile (`candex-app`)

Angular 20 + Ionic 8 + Capacitor 7, construite en **composants standalone** (pas de NgModules), structurée par feature, avec **NgRx** pour l'état.

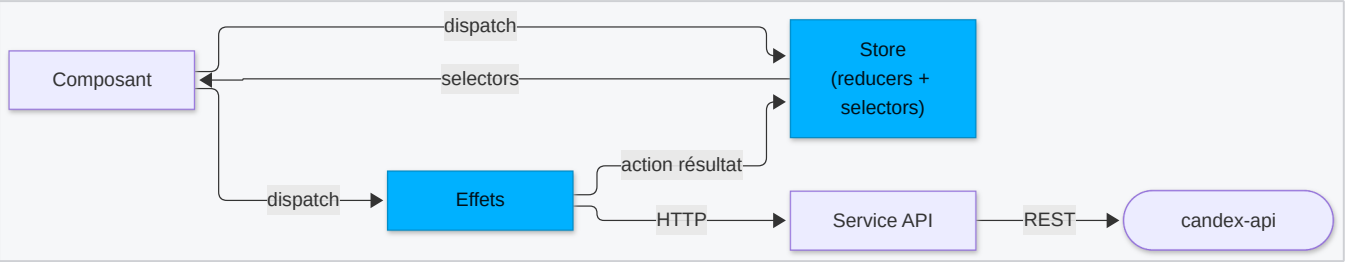
Couches & leurs rôles

Couche	Dossier	Rôle
Pages	`pages`	Écrans routés : le flux d'auth, le shell `tabs` (home / dex / profile), le détail d'une boisson, et les réglages
Composants	`components`	Briques d'UI réutilisables partagées entre les pages
Services	`services`	Effets de bord & intégrations : le client API (`api`), plus `auth`, `storage`, `theme`, `language`, `upload-care`, `haptics`, `native-app`, `dex-navigation` / `dex-ui`, `ui-alert`, et `app-initializer`
Store	`store`	État NgRx — une slice par domaine (voir plus bas)
Guards	`guards`	Protection des routes (`auth`, `app`) — tenir les non-authentifiés hors des écrans protégés
Interceptors	`interceptors`	`auth.interceptor` attache le token Firebase à chaque requête API
Pipes	`pipes`	Formatage de vue — ex. `dynamic-locale-date` pour les dates localisées

Le principe est le même qu'au back-end : **les composants affichent et dispatchent ; les services et le store font le travail**. Un composant n'appelle jamais l'API directement.

État avec NgRx

L'état est découpé en slices — `app`, `auth`, `dex`, `drink`, `preferences`, `user` — chacune une unité NgRx complète (`actions`, `reducer`, `effects`, `selectors`, `state`). Le flux est à sens unique :



Un composant **dispatche** une action ; un **reducer** met à jour la slice de façon pure, ou un **effect** appelle le service API et dispatche une action résultat ; les **selectors** renvoient le nouvel état aux composants. La persistance de l'état entre les lancements est assurée par `ngrx-store-localstorage` + Ionic Storage.

Navigation

Les routes sont standalone : le flux **auth** (`welcome`, `register`, `login`, `complete-profile`, `verify-email`, `two-factor`), le shell **tabs** (`home`, `dex`, `profile`), `drink-details/:drinkId`, et les **réglages** (`account`, `preferences`, `about`).

Préoccupations transverses

- **Localisation** — `ngx-translate` avec un service `language`, et un pipe `dynamic-locale-date` pour les dates.
- **Thème** — un service `theme` pilote le clair / sombre.
- **Natif** — plugins Capacitor (haptics, clavier), un service `native-app`, et Firebase Authentication via Capacitor.
- **Média** — upload d'images via Uploadcare (service `upload-care`) et recadrage via `ngx-image-cropper`.

Tableau de bord admin (`candex-dashboard`)

Angular 20 + Ionic 8 + **Angular Material**, pour gérer le catalogue. Il est volontairement plus simple que l'app : **état par services** (pas de NgRx), organisé en `components`, `pages`, `services`, `interceptors` et `interfaces`. Un interceptor attache les identifiants admin aux appels API, et il dialogue avec la même API REST. Sa console SQL s'appuie sur le rôle en lecture seule de l'API (voir Base de données & Modèle).

Où aller ensuite

- Look & feel, composants, tokens → **UI/UX (+ Design System)** *(à venir)*
- L'API que consomment les front-ends → Back-end
- La vue d'ensemble → Architecture & Vue d'ensemble

API / Contrats

Voici la référence de contrat de `candex-api`. Elle s'appuie sur Back-end (comment l'API est construite) et liste les endpoints par ressource. Le **contrat vivant et exhaustif** — schémas requête/réponse, exemples et essai en direct — est **Swagger UI** sur `/swagger-ui.html` ; cette page en est la carte stable.

Conventions

- **Enveloppe** — chaque réponse est un `Result<T>` : `{ success, data, error }` (voir Back-end).
- **Codes de statut** — issus d'exceptions typées (`404` / `400` / `409` / `422`…) ; voir Gestion d'erreurs du Back-end.
- **En-têtes** — les réponses de liste portent `X-Total-Count` ; les créations renvoient `201` avec un en-tête `Location`.
- **Authentification** — envoyez un token d'identité Firebase via `Authorization: Bearer <token>`.
- **Accès** — par endpoint, dans les tableaux : `USER_OR_ADMIN` = tout utilisateur authentifié, `ADMIN` = admin seulement, `PUBLIC` = sans auth, **own** = l'utilisateur authentifié n'agit que sur ses propres enregistrements.
- **Chemin de base** — les ressources sont servies sous la base de l'API (par exemple `/api/drinks`).

Catalogue

Drinks — `/drinks`

Méthode	Chemin	Accès	But
GET	/drinks	USER_OR_ADMIN	Lister les boissons
GET	/drinks/{id}	USER_OR_ADMIN	Une boisson
POST	/drinks	ADMIN	Créer une boisson
POST	/drinks/import	ADMIN	Import en masse
POST	/drinks/import/diff	ADMIN	Prévisualiser un import face au catalogue actuel
POST	/drinks/import/sync	ADMIN	Appliquer un import
PUT	/drinks/{id}/shift	ADMIN	Déplacer la position World Dex
PUT	/drinks/{id}/swap	ADMIN	Échanger deux positions World Dex
DELETE	/drinks/{id}	ADMIN	Supprimer une boisson

Cans — /cans

Méthode	Chemin	Accès
GET	/cans • /cans/{id}	USER_OR_ADMIN
POST	/cans	ADMIN
DELETE	/cans/{id}	ADMIN

Données de référence (lectures)

Les ressources de référence du catalogue exposent des lectures liste et par-id : **Brands** /brands, **Flavors** /flavors et **Flavor categories** /flavor-categories, **Colors** /colors

(+ `/colors/extract`), **Drink categories** `/drink-categories`, **Can formats** `/can-formats`, et **App versions** `/app-versions` (+ `/app-versions/current`).

World Dex — `/world-dex`

Méthode	Chemin	Accès	But
GET	`/world-dex/drinks/{id}/history`	USER_OR_ADMIN	Historiqu de positi d'une boisson
GET	`/world-dex/drinks/{id}/stats`	USER_OR_ADMIN	Stats De d'une boisson
GET	`/world-dex/snapshot`	USER_OR_ADMIN	Instantar du Dex
GET	`/world-dex/movers`	USER_OR_ADMIN	Mouvem récents
GET	`/world-dex/positions/{position}/occupants`	USER_OR_ADMIN	Qui occu une posi

Utilisateurs & collection

Users — `/users`

Méthode	Chemin	Accès	But
GET	/users	ADMIN	Lister les utilisateurs
GET	/users/{id}	own	Un utilisateur
GET	/users/firebase/{firebaseUid}	own	Un utilisateur par UID Firebase
GET	/users/{id}/drinks	own	Les boissons débloquées d'un utilisateur
GET	/users/exists	PUBLIC	Vérifier l'existence
POST	/users	PUBLIC	S'inscrire
PUT • PATCH	/users/{id}	own	Mettre à jour un utilisateur
DELETE	/users/{id}	own	Supprimer un utilisateur

Unlocked drinks — /user-unlocked-drinks

Méthode	Chemin	Accès	But
GET	/user-unlocked-drinks/{userId}	own	Les déblocages d'un utilisateur
POST	/user-unlocked-drinks	own	Débloquer une boisson
DELETE	/user-unlocked-drinks/{userId}/{drinkId}	own	Retirer un déblocage

Admin & opérations

- **Console SQL** — `POST /admin/...` (ADMIN) : exécuter une requête en lecture seule via le rôle dédié (voir Base de données & Modèle).
- **Reports** — `POST /reports` : générer un rapport.

Le flux d'import en masse

Importer un lot de boissons est un pipeline en trois étapes, réservé aux admins :

1. `POST /drinks/import` — soumettre le payload.
2. `POST /drinks/import/diff` — prévisualiser les changements face au catalogue actuel.
3. `POST /drinks/import/sync` — les appliquer.

Où aller ensuite

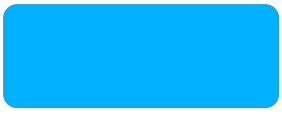
- Comment l'API est construite → Back-end
- Schémas vivants & essai en direct → **Swagger UI** (`/swagger-ui.html`)

UI / UX

L'interface de Candex est une app mobile **Ionic 8 + Angular 20**. Ce handbook couvre le côté design et interaction ; pour la structure du code front-end, voir Front-end.

Design tokens

Les couleurs, la typographie et les surfaces sont définies une seule fois sous forme de **variables CSS Ionic** et réutilisées partout, pour que la marque reste constante sur chaque écran.



primary

#00B1FF

Bleu Candex — actions principales, accents

secondary

#ffffff

Blanc



tertiary

#033C44

Teal profond — emphase secondaire



success

#2dd36f

État succès



warning

#ffc409

État avertissement



danger

#eb445a

État danger / erreur

Chaque couleur est livrée avec ses variantes `-shade`, `-tint` et `-contrast`, pour que le texte respecte toujours son fond. La typographie est **Rubik**, auto-hébergée depuis `assets/fonts` (Regular, Medium, Bold et le reste de la famille) et branchée via `--ion-font-family`, donc l'app s'affiche identiquement hors-ligne et sur toutes les plateformes.

Typographie

La police de l'app est **Rubik**, auto-hébergée depuis `assets/fonts` et définie via `--ion-font-family`. Trois graisses sont chargées et portent toute la hiérarchie : **Regular (400)**

pour le corps de texte, **Medium (500)** pour l'emphase et les labels, et **Bold (700)** pour les titres et le texte fort.

Les tailles de texte suivent la **typographie intégrée d'Ionic** (l'app importe `typography.css`) plutôt qu'une échelle sur mesure : une base de **16px (1rem)**, les titres décroissant de `<h1>` à `<h6>` sur l'échelle par défaut d'Ionic. En bref, l'app possède la *famille* ; Ionic possède les *tailles* — ce qui garde le texte constant et respecte les réglages d'accessibilité (dynamic type) de la plateforme.

Thèmes — Light, Dark, System

L'app propose trois thèmes via un `ThemeService` : **Light**, **Dark** et **System** (qui suit le réglage de l'appareil). Le choix est appliqué en posant un attribut `theme` sur `<body>`, et les blocs de tokens `body[theme=light]` / `body[theme=dark]` échangent la palette en dessous — chaque composant réagit automatiquement puisqu'il lit les mêmes variables.

Bibliothèque de composants

Au-dessus des composants d'Ionic (boutons, cards, listes, modales, tabs) s'ajoute un ensemble de **composants partagés** qui portent les patterns propres à l'app :

- **Contenu** — `drink-card` affiche une canette de façon constante dans le Dex et les listes.
- **Feedback** — `banner` et `error-banner` remontent des messages inline ; `form-errors` affiche la validation au niveau du champ sous les inputs.
- **Tâches ciblées (modales)** — `avatar-cropper-modal`, `fullscreen-image-modal`, `report-problem-modal` et les flux de compte Google (`google-account-modal`, `google-link-prompt-modal`, `google-photo-choice-modal`) gardent une seule tâche en vue à la fois.
- **Auth** — `social-auth-buttons` fournit les points d'entrée de connexion sociale.

Les icônes viennent d'**Ionicons**, le set fourni avec Ionic.

Navigation

L'app repose sur un **shell à trois tabs** (`tabs.page`) :

- **Dex** — la collection, et le tab d'atterrissage par défaut.
- **Search** — une action au tap qui ouvre la recherche plutôt qu'une page de tab statique.
- **Profile** — le compte et les réglages de l'utilisateur.

Les pages sont des **composants standalone chargés en lazy** (`loadComponent`), donc chaque écran se charge à la demande. Hors des tabs, une **stack d'auth** porte le parcours de connexion — `welcome`, `register`, `login`, `verify-email`, `two-factor` et `complete-profile` — et des routes dédiées gèrent les vues de détail et de réglages (`drink-details/:drinkId`, `settings`). L'authentification elle-même est traitée dans Sécurité.

Conventions d'interaction & de contenu

Quelques patterns reviennent partout :

- **Des modales pour le travail ciblé** — recadrer un avatar, voir une image en plein écran, lier un compte Google ou signaler un problème se font dans une modale, en gardant l'écran sous-jacent intact.
- **Validation inline, au niveau du champ** — les formulaires affichent les erreurs à côté du champ fautif via `form-errors`, pas en alertes intrusives.
- **Des banners pour le statut** — les messages transitoires et d'erreur apparaissent en banners plutôt qu'en dialogues bloquants.

Internationalisation

L'UI est entièrement traduite avec **ngx-translate** : les chaînes sont externalisées dans `en.json` et `fr.json` et rendues via le pipe `| translate` (`tabs.dex`, `tabs.search`, …), pour que l'anglais et le français restent synchronisés et qu'une nouvelle langue se résume à ajouter un fichier de locale.

Pour aller plus loin

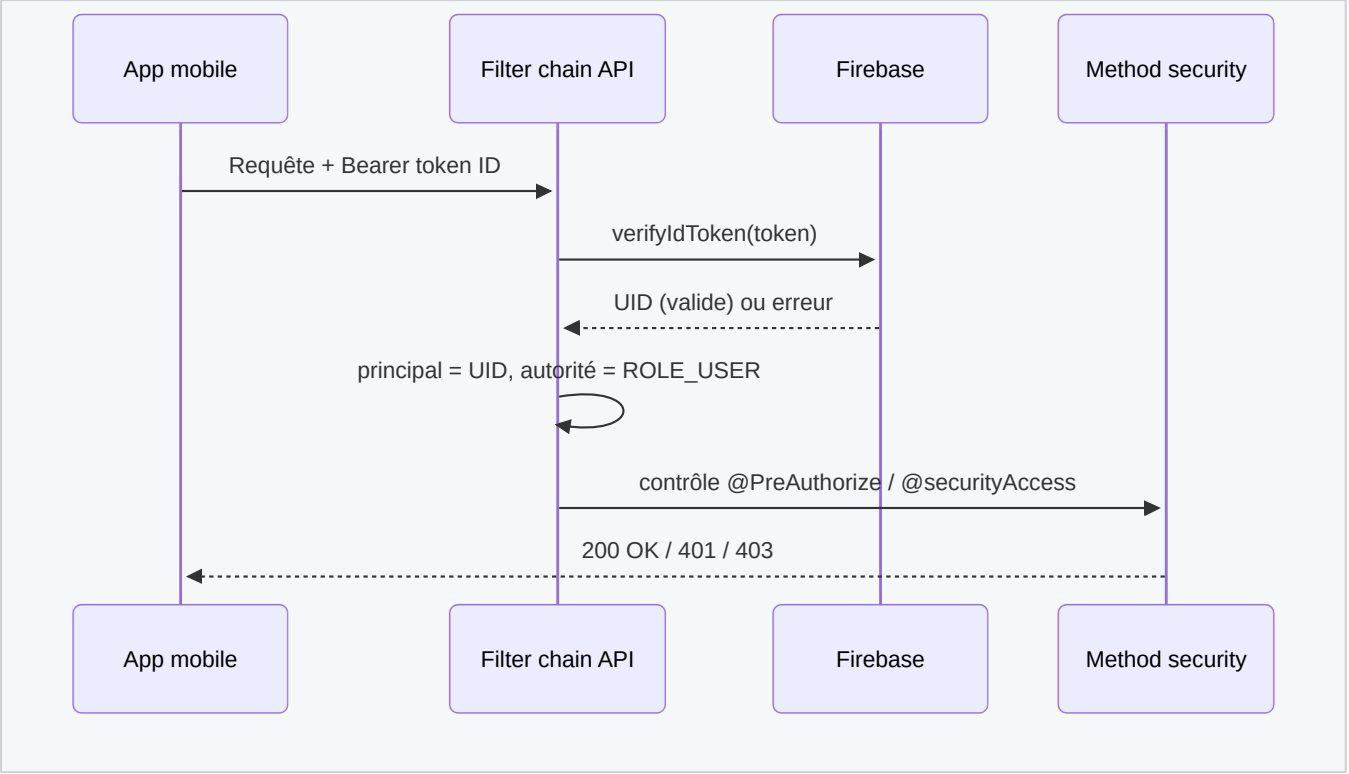
- Le code derrière ces écrans → Front-end

- À quoi sert l’app → Comprendre Candex
- Connexion et permissions → Sécurité

Sécurité

La sécurité est imposée dans l'**API**. L'authentification décide *qui* appelle ; l'autorisation décide *ce qu'il peut faire. La couche HTTP reste volontairement fine — elle définit le mécanisme d'authentification et quelques routes publiques — tandis que les vraies règles métier vivent dans la method security, à côté des endpoints qu'elles protègent.

L'authentification d'une requête en un coup d'œil



Authentification

Deux mécanismes coexistent.

Les **tokens Firebase** sont la voie principale, pour les utilisateurs de l'app. Une requête porte `Authorization: Bearer <token>` ; le `FirebaseAuthenticationFilter` le vérifie (`FirebaseAuth.verifyIdToken`) et, en cas de succès, pose une authentification Spring Security dont le **principal est l'UID Firebase** et l'autorité `ROLE_USER`. La vérification est non bloquante : si le token est absent, invalide, expiré ou révoqué, le filtre vide simplement

le contexte et laisse l'autorisation rejeter la requête en aval. Firebase est la source de vérité pour l'identité applicative.

Une **Basic Auth HTTP interim** couvre l'administrateur. Un magasin d'utilisateurs en mémoire (`InMemoryUserDetailsManager`) contient deux comptes chargés depuis des variables d'environnement — un `user` standard (`ROLE_USER`) et un `admin` (`ROLE_ADMIN`) — avec des mots de passe hashés en BCrypt. Le code la documente comme un pont temporaire, destiné à être remplacé plus tard par une authentification JWT.

La filter chain est **stateless** (`SessionCreationPolicy.STATELESS`), le **CSRF est désactivé** (il n'y a ni session serveur ni cookie à protéger), et le **CORS** est géré par un `CorsFilter` dédié avec une liste d'origines autorisées fixe (`CorsOrigins`).

Rôles

Il y a deux autorités — `ROLE_USER` et `ROLE_ADMIN` (`SecurityRoles`).

Autorisation

L'autorisation s'exécute sur deux couches.

À la couche HTTP (`SecurityConfig`), seules quelques règles sont déclarées :

- **Public** — `/error`, `/actuator/health`, l'inscription (`POST /users`) et le test d'existence (`GET /users/exists`).
- **Admin uniquement** — l'UI Swagger et la doc OpenAPI (`/swagger-ui/**`, `/v3/api-docs/**`) et l'actuator (`/actuator/**`).
- **Tout le reste** — authentifié.

À la couche méthode (`@EnableMethodSecurity`), chaque endpoint porte une règle `@PreAuthorize`, construite à partir de `SecurityExpressions` réutilisables :

- `USER_OR_ADMIN` — tout utilisateur authentifié ou admin.
- `ADMIN_ONLY` — autorité admin requise (écritures du catalogue, imports, opérations d'admin).
- `PERMIT_ALL` — explicitement ouvert.

Les endpoints liés à la propriété passent par le bean `@securityAccess` (`SecurityAccess`) plutôt que par un rôle seul, p. ex. `@PreAuthorize("@securityAccess.canAccessUserId(authentication, #id)")`. La règle est constante : un **admin contourne la propriété**, sinon l'UID Firebase (ou l'id utilisateur) de l'appelant doit correspondre au propriétaire de la ressource. Le même bean garde aussi l'accès par UID Firebase (`canAccessFirebaseUid`) et la gestion des boissons débloquées (`canManageUserUnlockedDrink`).

Secrets

Aucun credential ni clé n'est versionné. Le compte de service Firebase est chargé à l'exécution depuis un chemin de fichier configuré (`candex.firebase.config-path`), jamais depuis le dépôt. Les comptes Basic Auth (`candex.security.*`) et les credentials de la base viennent de variables d'environnement, par profil Spring — les dépôts ne livrent que des fichiers `.env.example` avec des placeholders. Voir Installation locale pour les définir en local et Déploiement pour les environnements déployés.

Côté client

L'app mobile authentifie les utilisateurs avec **Firebase Authentication** et appelle l'API avec le token d'ID obtenu dans l'en-tête `Authorization` ; l'API fait confiance à Firebase pour avoir authentifié l'utilisateur et dérive l'identité du token vérifié. Voir Architecture pour la façon dont les pièces s'assemblent.

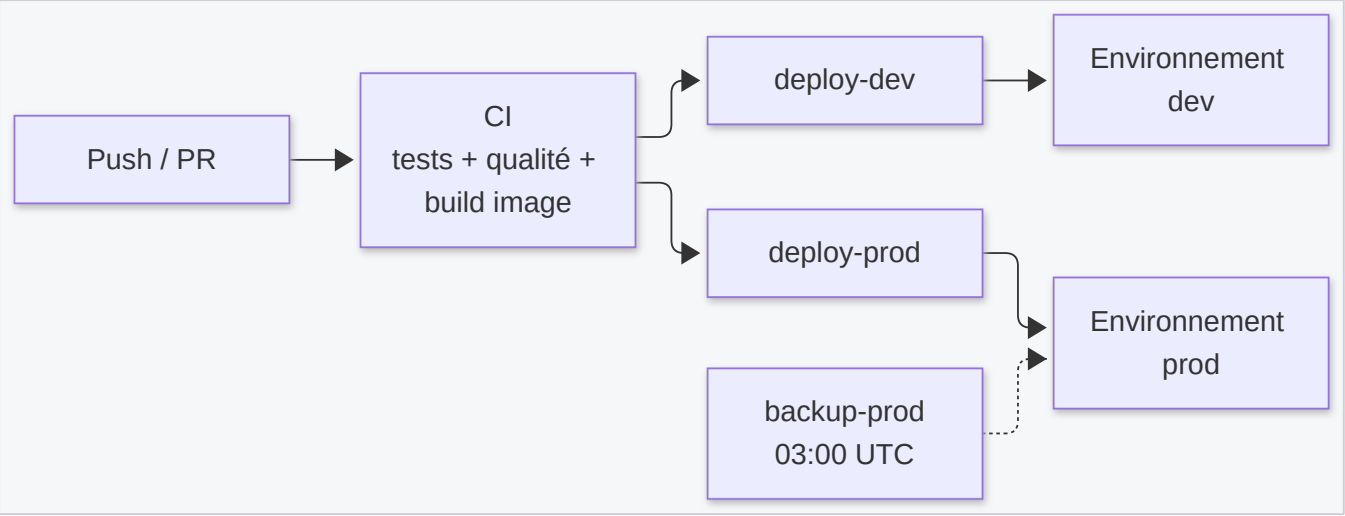
Pour aller plus loin

- Les endpoints que ces règles protègent → API / Contrats
- Les couches derrière → Back-end
- La vue d'ensemble → Architecture

Déploiement

Voici comment Candex est construit, livré et exploité. L'API tourne en stack **Docker Compose** par environnement ; l'app mobile est livrée sur les stores avec **Fastlane**. Pour la lancer sur votre machine, voir Installation locale ; pour le système autour, Architecture & Vue d'ensemble.

Le pipeline en un coup d'œil



Environnements

Chaque environnement est sa propre stack Compose (un conteneur API + un conteneur PostgreSQL 17) :

Environnement	Fichier compose	Exposition
local	`compose.local.yaml`	Stack jetable via `scripts/candex.sh` (Installation locale)
dev	`compose.dev.yaml`	`api-dev` + `db-dev` ; API sur l'hôte `8081`, debug sur `5005`
prod	`compose.prod.yaml`	`api-prod` + `db-prod` ; **aucun port publié** — atteint via le reverse proxy

Chaque conteneur API tourne avec `SPRING_PROFILES_ACTIVE` réglé sur son environnement, redémarre `unless-stopped`, et garde des logs `json-file` rotés.

L'image de l'API

L'API est construite par un `Dockerfile` multi-stage — compilée avec Maven, puis exécutée sur un JRE léger :

```
FROM maven:3.9.9-eclipse-temurin-21 AS build
# ... mvn -B -DskipTests package
FROM eclipse-temurin:21-jre
COPY --from=build /app/target/*.jar /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

La configuration est entièrement pilotée par l'environnement (base, sécurité, console SQL, Uploadcare) — voir Installation locale pour les variables. Aucun secret n'est intégré à l'image.

Exploiter une stack

`scripts/candex.sh` opère une stack dev ou prod en cours d'exécution depuis le serveur — sans l'arrêter :

```
./scripts/candex.sh psql <dev|prod>        # une session psql
./scripts/candex.sh logs <dev|prod>        # suivre les logs de l'API
./scripts/candex.sh dump <dev|prod>        # pg_dump de la base
./scripts/candex.sh exec <dev|prod> "<SQL>"
```

CI/CD

Des GitHub Actions pilotent l'intégration et la livraison (`.github/workflows`) :

Workflow	Rôle
`ci.yml`	Garde-fou qualité sur push / PR — build, tests, lint (et Qodana)
`deploy-dev.yml` • `deploy-prod.yml`	Déployer l'API en dev / prod
`backup-prod.yml`	Sauvegarde planifiée de la base de production

Le dashboard a son propre garde-fou `ci.yml`.

Reverse proxy & TLS

> **À confirmer**
>
> L'API de production ne publie aucun port ; le point d'entrée public et le TLS sont gérés par un **reverse proxy au niveau serveur** (une variable `CADDY_EMAIL` pointe vers Caddy). La configuration exacte du proxy vit hors de ces dépôts et n'est pas encore documentée ici.

Releases de l'app mobile

L'app est livrée sur le **Play Store** et l'**App Store** avec **Fastlane**, déclenchée manuellement depuis GitHub Actions. Le bump de version et le changelog restent locaux et relus par un humain — la CI ne déploie qu'un tag existant :

```
npm run release 1.2.0            # bump + tag annoté x.y.z (message = changelog)
git push && git push origin 1.2.0
# GitHub ▸ Actions ▸ « Deploy to Stores » ▸ Run workflow (store : both / play / appstore)
```

`deploy.yml` (un `workflow_dispatch` manuel) construit le tag choisi et le soumet via les lanes Fastlane `android deploy` (AAB → Play production) et `ios deploy` (IPA → App Store Connect).

Où aller ensuite

- Sauvegardes de la base → Sauvegardes
- La lancer sur votre machine → Installation locale
- L'API elle-même → Back-end

Sauvegardes

Candex sauvegarde sa base **PostgreSQL de production**. Ce handbook couvre la stratégie, les scripts, comment restaurer, et comment c'est automatisé. Pour où vit la base, voir Base de données & Modèle ; pour les environnements, Déploiement.

> **Note**
>
> Le dépôt `candex-backups` est l'ancien outil de l'ère Firestore et ne sauvegarde plus de vraies données. Les sauvegardes actuelles sont des dumps PostgreSQL pilotés depuis `candex-api/scripts/backup/`.

Stratégie

Seule la **prod** (`candex_prod`) est sauvegardée — la dev sert de terrain d'essai. L'approche : un dump complet quotidien avec rétention grandfather-father-son (GFS), plus un instantané permanent par release :

- **Quotidien** — dumps complets — `KEEP_DAILY = 7`
- **Hebdomadaire** — le dump du dimanche est promu — `KEEP_WEEKLY = 5`
- **Mensuel** — le dump du 1er du mois est promu — `KEEP_MONTHLY = 12`
- **Par version** — un instantané labellisé (`--label x.y.z`), **gardé à vie, jamais élagué**

Sauvegardes complètes — `daily_backup.sh`

Le script principal exécute un `pg_dump -Fc` (format custom : compressé et restaurable sélectivement) de toute la base, à travers le conteneur Postgres en cours :

```
./scripts/backup/daily_backup.sh prod                 # dump complet quotidien
./scripts/backup/daily_backup.sh prod --label 1.1.9   # + un instantané de version conservé
```

Chaque exécution écrit dans `backups/<env>/full/`, valide le dump (non vide et passant `pg_restore --list`), puis le promeut dans les paliers `daily`, `weekly`, `monthly` et `version` selon le cas :

```
backups/prod/full/
├── daily/     candex_prod_<timestamp>.dump
├── weekly/    (dimanches)
├── monthly/   (1er du mois)
└── version/   candex_prod_v1.1.9.dump   (gardé à vie)
```

Le dossier `backups/` est git-ignoré — les dumps ne vont jamais dans git.

Exports sélectifs — `dump_*.sh`

En plus des dumps complets, des exports **data-only** par domaine couvrent les données runtime difficiles à reconstruire — `dump_catalog_data.sh`, `dump_dex_data.sh`, `dump_user_data.sh` (et `dump_all_runtime_data.sh`). Les données de référence statiques sont volontairement exclues, car Flyway les seede (voir Base de données & Modèle).

Restauration & test

Une sauvegarde ne vaut que par une restauration testée. `restore.sh` restaure un dump dans une base **jetable** pour la vérifier, puis la supprimer :

```
./scripts/backup/restore.sh <fichier.dump> candex_restore_test
# vérifier, puis : dropdb candex_restore_test
```

`pg_restore` doit être de la même version que le serveur ou plus récent.

Automatisation (CI)

Les sauvegardes tournent sur le runner self-hosted présent sur le serveur de production, via GitHub Actions :

Workflow	Quand	Quoi
`backup-prod.yml`	Quotidien à **03:00 UTC** (et manuel)	`daily_backup.sh prod`
`deploy-prod.yml`	Avant chaque déploiement prod	`daily_backup.sh prod --label <version>` — un point de reprise pré-déploiement

Aucun cron OS nécessaire — la planification vit dans le workflow.

Stockage & hors-site

Les dumps sont gardés **sur le serveur** pour l'instant ; les sortir de l'hôte (un second disque ou `rsync` vers une autre machine) est une amélioration prévue. Une variante autonome sous `ops/backup/` (un timer `systemd` plus copie hors-site `rsync` et alerting en cas d'échec) est disponible pour ce setup côté serveur.

Conventions

- **Aucun secret dans git** — le mot de passe Postgres vit dans `~/.pgpass` (`chmod 600`) ; la config locale (`backup.conf`) et les dumps `backups/` ne sont jamais committés.
- **Complet + sélectif** — les dumps complets `-Fc` sont le chemin de restauration ; les exports par domaine sont des instantanés de confort.
- **Tester les restaurations** — un dump jamais restauré n'est pas une sauvegarde.

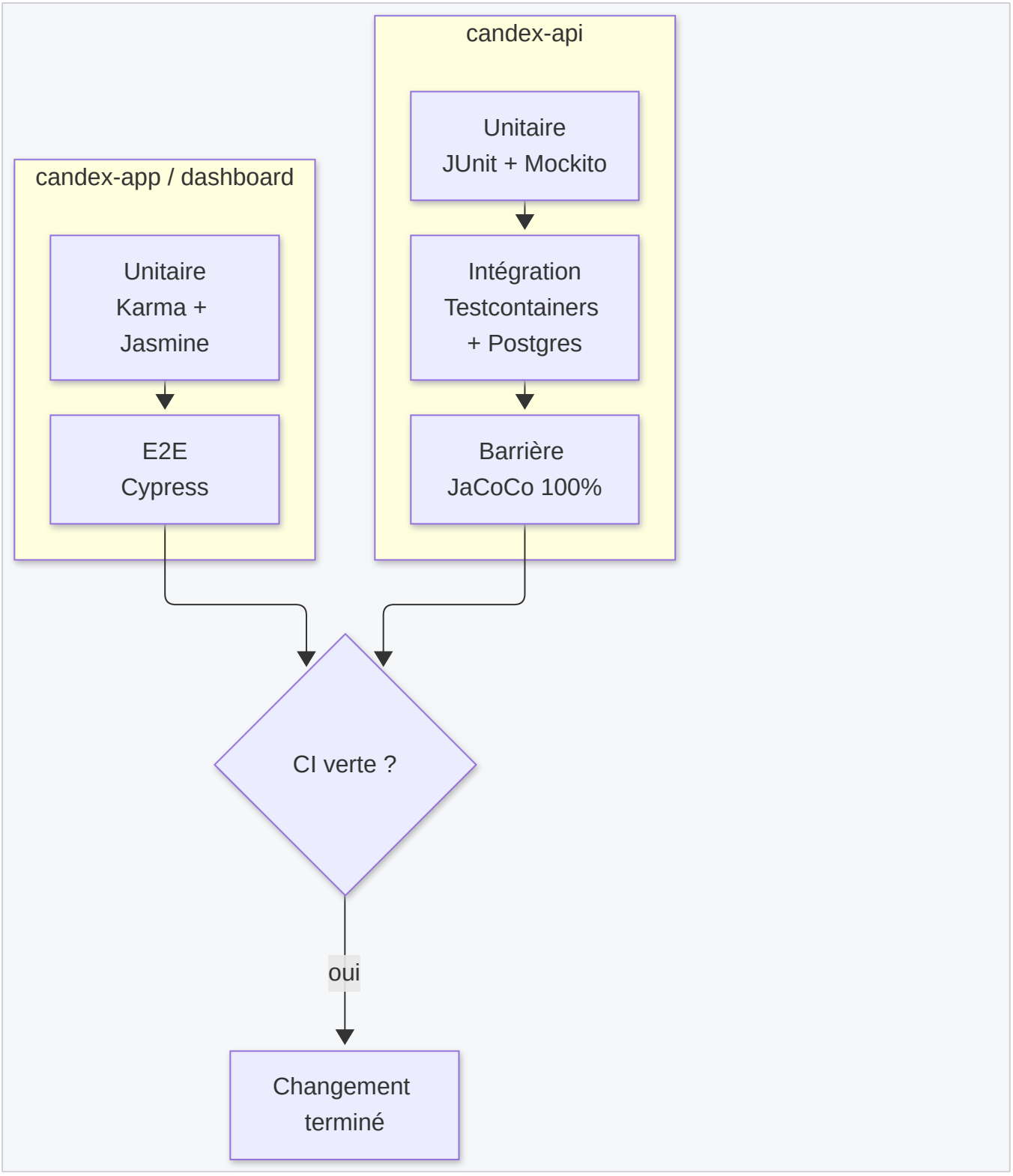
Où aller ensuite

- Où vivent les données → Base de données & Modèle
- Environnements & CI → Déploiement

Tests

Les tests couvrent toute la stack et sont imposés en CI. Pour l’organisation du code, voir Back-end et Front-end.

Le dispositif de test en un coup d’œil



Back-end (`candex-api`)

L'API compte environ **90 classes de test** sur deux niveaux :

- **Tests unitaires** — **JUnit 5** avec **Mockito** pour la logique isolée.

- **Tests d'intégration** — **Testcontainers** démarre un vrai conteneur **PostgreSQL**, de sorte que les tests tournent contre la vraie base plutôt que contre un mock.

La couverture est une barrière stricte : **JaCoCo** impose **100 %** au niveau du bundle (lignes et branches) — le build échoue sous le seuil. **Checkstyle** impose le style de code.

```
./mvnw test        # lance les tests
./mvnw verify      # tests + barrière de couverture JaCoCo + Checkstyle
```

Front-ends (`candex-app`, `candex-dashboard`)

Les deux applications Angular utilisent **Karma + Jasmine** pour les tests unitaires, lancés en mode headless avec couverture. Les specs vivent à côté du code (`*.spec.ts`) — environ **80** dans l'app et **40** dans le dashboard. L'app mobile a aussi **Cypress** pour les tests de bout en bout.

```
npm run test       # mode watch, avec couverture
npm run test:ci    # headless (ChromeHeadless), une passe, avec couverture
```

Intégration continue

Chaque push et chaque pull request lance un **quality gate** (`.github/workflows/ci.yml`) par dépôt :

Dépôt	Ce que la CI exécute
`candex-api`	`mvn clean verify checkstyle:checkstyle` (tests + barrière JaCoCo 100 % + Checkstyle), puis un build d'image Docker ; publie les rapports Checkstyle et JaCoCo
`candex-app` • `candex-dashboard`	`ng test` headless avec couverture, plus le lint

Qodana ajoute une analyse statique de qualité de code par-dessus.

Conventions

Les tests épousent le code qu'ils couvrent, à chaque niveau :

- **Une unité par fichier** — une classe de test (API) ou un `describe` racine (front) par classe ou service, nommé d'après elle : `@DisplayName("ColorMatcher")`, `describe('SettingsActionsService')`.
- **Une méthode par groupe** — chaque méthode testée a sa propre classe `@Nested` côté API (`@DisplayName("findNearestAnchor()")`) ou un `describe('refreshProviderState()')` imbriqué côté front.
- **Un cas par test** — un seul `@Test` / `it` par scénario, nommé comme le comportement qu'il vérifie (*« should return semantic match when a ColorGroup zone matches »*, *« dispatches linkGoogle when not yet linked »*). Côté API chaque niveau porte un `@DisplayName`, si bien que le rapport de tests se lit comme des phrases.
- **Un smoke test par fichier** — `contextLoads()` (*« should load correctly »*) côté API, `it('should be created')` côté front, pour qu'une erreur de câblage ou d'injection remonte en premier.
- **Des fixtures partagées, pas d'objets recréés** — les données de test réutilisables vivent à un seul endroit et sont importées, jamais reconstruites dans chaque fichier : le package `util/test/fixture` de l'API (`TestApp`, `TestEntity`, `TestGetDTO`, `TestPostDTO` — classes `final` exposant des constantes figées comme `FIXED_NOW` et des entités/DTOs préparés), et côté front `utils/tests/mocks.ts` et les test helpers (dashboard : `tests/mocks`).

Et sur toute la stack :

- Les tests vivent **à côté du code** qu'ils couvrent (`*.spec.ts` ; `*Test` sur l'API).
- Les tests d'intégration utilisent **Testcontainers**, pas des mocks : ils exercent du vrai SQL et les vraies migrations.
- La barrière de **100 % de couverture** de l'API implique que tout nouveau code arrive avec ses tests — et un changement n'est pas terminé tant que la CI n'est pas verte.

Pour aller plus loin

- Style de code, conventions, analyse statique → Qualité logicielle
- L'API → Back-end • Les front-ends → Front-end

Qualité logicielle

La qualité ici, c'est du code qui reste constant, simple et peu complexe — imposé par l'outillage en CI, pas par la mémoire. Les tests et la couverture sont traités à part dans Tests ; ce handbook parle de **style, conventions et analyse statique**.

Back-end — Checkstyle

L'API est vérifiée contre une config **Checkstyle** sur mesure (`checkstyle.xml`) couvrant le nommage (`TypeName`, `MethodName`, `MemberName`, `ConstantName`), les imports (pas d'import en étoile, pas d'import inutile ou redondant), les espaces, les accolades (`NeedBraces`), l'ordre des modificateurs et l'indentation. Les lignes sont plafonnées à **120** caractères.

Elle plafonne aussi la complexité, pour que les méthodes restent courtes et lisibles :

Règle	Limite
Longueur de méthode	60 lignes
Paramètres par méthode	7
Complexité cyclomatique	10
Complexité NPath	200
Longueur d'une classe interne anonyme	20 lignes

Un `checkstyle-suppressions.xml` associé relâche quelques règles là où elles ajoutent du bruit : les contrôleurs sont dispensés du Javadoc sur méthodes (les annotations Spring parlent d'elles-mêmes), les entités peuvent utiliser des nombres magiques, et les mappers MapStruct générés, les contrôleurs et les sources de test sont exemptés des contrôles de longueur de ligne et d'indentation.

Checkstyle tourne en CI (`mvn … checkstyle:checkstyle`) et le rapport HTML est publié en artefact de build.

Front-ends — ESLint & Prettier

Les deux applications Angular passent le lint avec **ESLint 9** via `@angular-eslint` et `@typescript-eslint`, et le formatage avec **Prettier** :

```
npm run lint      # ng lint (ESLint)
npm run format    # prettier --write .
```

`npm run lint` tourne en CI pour l'app comme pour le dashboard. Un `.editorconfig` fixe les bases communes — UTF-8, indentation à **2 espaces**, une ligne finale, les espaces de fin supprimés, et les **guillemets simples** en TypeScript. Le dashboard ajoute quelques plugins ESLint supplémentaires (`import`, `jsdoc`, `prefer-arrow`). L'app mobile dispose de **husky** pour les hooks Git, même si le lint tourne pour l'instant manuellement et en CI plutôt qu'au commit.

Analyse statique

Deux couches d'analyse statique s'ajoutent au lint.

DeepSource tourne en continu sur les trois dépôts (`.deepsource.toml`) :

- **API** — l'analyzer Java (runtime 21), avec `test_patterns` restreint à `**/*Test.java` pour que les fixtures partagées (`TestApp`, `TestEntity`, `TestGetDTO`, `TestPostDTO`, `TestHelper`) ne soient pas signalées comme fichiers de test vides.
- **App mobile** — l'analyzer JavaScript avec le plugin Angular (environnements Node, Jasmine, Cypress) ; deux règles sont ignorées là où les patterns Angular les déclenchent à tort (classes de module vides `JS-0327`, aliasing d'input requis `JS-0585`).
- **Dashboard** — l'analyzer JavaScript avec le plugin Angular (Node, Jasmine).

JetBrains Qodana ajoute une seconde passe (le profil `qodana.starter`), mais seulement sur l'**API** (`qodana_code_quality.yml`, linter JVM community). Côté app mobile un workflow Qodana existe mais est **désactivé** (`.yml.disabled`), et le dashboard n'en a pas.

Les seuils du quality gate de Qodana dans `qodana.yaml` sont laissés en commentaire, donc il remonte ses findings sans faire échouer le build sur un décompte *(à confirmer)*.

Où la qualité est imposée

Dépôt	Style	Analyse
`candex-api`	Checkstyle	DeepSource + Qodana
`candex-app`	ESLint + Prettier	DeepSource
`candex-dashboard`	ESLint + Prettier	DeepSource

L'API a aussi la barrière JaCoCo 100 % — voir Tests. Le lint et Checkstyle tournent en CI à chaque push et pull request, DeepSource analyse chaque changement en continu, et un changement n'est pas terminé tant que tout n'est pas vert. Pour la vue complète de la CI et les barrières tests/couverture, voir Tests.

Pour aller plus loin

- Tests et couverture → Tests
- Le code auquel ces règles s'appliquent → Back-end • Front-end

Glossaire

Candex a son propre vocabulaire. Voici les termes partagés par l'app, la Base de données et le Back-end, ancrés dans le modèle de données.

Collection & progression

Dex — la **collection personnelle** d'un utilisateur : l'ensemble des boissons qu'il a débloquées. C'est le tab d'accueil de l'app et le cœur de l'expérience.

Unlock (débloquer) — l'action d'ajouter une boisson à son Dex. Une boisson ne peut être débloquée qu'une fois par utilisateur (`UserUnlockedDrink`, clé utilisateur + boisson), et le déblocage est horodaté.

World Dex — la **liste globale et classée** des boissons, partagée par tout le monde : chaque boisson occupe une position numérotée dans le dex mondial. À distinguer du Dex personnel d'un utilisateur.

Historique du World Dex — l'enregistrement de l'évolution des positions du World Dex. Un trigger de base de données journalise chaque *départ* (la position qu'une boisson a quittée) et une ligne d'*arrivée* marque sa première entrée, de sorte que la position d'une boisson à n'importe quel instant passé peut être reconstruite. Voir Base de données pour le mécanisme.

Catalogue

Drink (boisson) — l'élément central du catalogue : un produit de boisson précis, avec ses noms, sa marque et sa catégorie, ses saveurs, ses flags d'édition, ses dates de cycle de vie, et la version de l'app où il a été introduit. La nutrition et les claims sont stockés à part, par pays.

Can (canette) — une canette physique d'une boisson, décrite par son format, sa couleur, son pays et sa langue. Une boisson peut avoir plusieurs canettes.

Brand (marque) — la marque de boisson à laquelle appartient une boisson (le fabricant).

Series (série) — une sous-ligne au sein d'une marque — une sous-famille de catalogue, p. ex. la gamme *Ultra* de Monster.

Category (catégorie de boisson) — le type de boisson (la classification sous laquelle tombe une boisson).

Flavor (saveur) — un tag de saveur attaché à une boisson ; une boisson peut en porter plusieurs. Les saveurs sont regroupées en catégories de saveurs et ont des noms localisés (et des descriptions optionnelles) par langue.

Edition (édition) — des flags d'édition sur une boisson, distinguant les éditions spéciales ou limitées du produit standard.

Predecessor (prédécesseur) — un lien d'une boisson vers celle qu'elle remplace, capturant la lignée d'un produit à travers reformulations ou rebrands.

Données localisées

Certaines données d'une boisson varient selon la région, donc elles sont stockées par pays (et parfois par langue) plutôt que sur la boisson elle-même.

Localization (localisation) — le nom marketing localisé d'une boisson et un texte descriptif optionnel pour une combinaison pays + langue (`DrinkLocalization`).

Composition — la composition nutritionnelle d'une boisson pour un pays donné, toutes les valeurs exprimées **pour 100 mL** (`DrinkComposition`).

Claim — une allégation diététique ou de certification d'une boisson dans un pays précis (`DrinkClaim`), p. ex. un label diététique ou réglementaire.

Données de référence

Valeurs de référence partagées, utilisées dans tout le catalogue :

Country (pays) · Language (langue) — les dimensions régionales sur lesquelles sont clés les données localisées.

Can format (format de canette) — une taille/un format de canette standard.

Color (couleur) — une couleur servant à identifier le design d'une canette.

App version (version de l'app) — une version publiée de l'app ; les éléments du catalogue enregistrent la version où ils ont été *introduits*, ce qui pilote aussi la timeline du World Dex.

Personnes

User (utilisateur) — un titulaire de compte. Un utilisateur possède un Dex (ses boissons débloquées) et est authentifié via Firebase — voir Sécurité.

Pour aller plus loin

- Comment tout cela se traduit en tables → Base de données
- Comment les couches les utilisent → Back-end
- Ce que l'app en fait → Comprendre Candex